

# CaptionAI: AI-Powered Social Media Caption Generator

## Project Documentation & Implementation Manual

Team ID: SWTID-2026-9692

SmartBridge Skill Wallet Project

### 1. Project Overview & Description

**CaptionAI** harnesses Generative AI to provide intelligent social media content generation, offering users a fast and personalized caption-creation experience. The platform incorporates three main features in a unified workflow:

- **Caption Generator:** Produces three unique, platform-optimized captions per request (e.g., emoji-rich conversational copy for Instagram, insight-driven copy for LinkedIn).
- **Hashtag Suggester:** Generates ten relevant hashtags blending high-reach broad tags with targeted niche tags.
- **Call-to-Action (CTA) Generator:** Delivers three punchy, action-oriented phrases tailored to the selected tone and platform.

Utilizing Groq's API with the `llama-3.3-70b-versatile` model, CaptionAI processes user inputs in seconds. Built with Flask and deployed publicly via Ngrok, the platform ensures a seamless experience with secure API key handling via `.env` environment configurations.

### 2. Deployment Scenarios

#### Scenario 1: E-Commerce Product Launch (Instagram / Casual)

A user describes a new summer collection of sustainable sunglasses. CaptionAI produces 3 emoji-rich captions optimized for visual storytelling, 10 mixed hashtags (#SustainableFashion, #SummerVibes), and 3 punchy CTAs.

#### Scenario 2: B2B SaaS Product Promotion (LinkedIn / Professional)

A marketer promotes an enterprise cloud tool. CaptionAI delivers thought-leadership style captions, industry hashtags, and CTAs focused on driving demo sign-ups and whitepaper downloads.

#### Scenario 3: Retail Flash Sale (Instagram / Promotional)

A user running a weekend sale inputs offer details. The AI generates urgency-driven captions with power words, deal-targeted hashtags, and CTAs like "Shop before it's gone!"

### 3. Technical Architecture & Prerequisites

CaptionAI utilizes a modular three-tier architecture:

- **Frontend:** Responsive single-page glassmorphism interface built with HTML, CSS, and Vanilla JavaScript.

- **Backend:** Flask-based web application orchestrating data validation, prompt assembly, and route handling.
- **AI Integration:** Groq API delivering ultra-fast inference with the LLaMA 3.3-70B Versatile model.

## Prerequisites & Dependencies

- Python 3.8+ & Pip package manager
- Flask framework ( `Flask==3.0.3` )
- Groq Python SDK ( `groq==0.11.0` )
- Environment variables management ( `python-dotenv==1.0.1` )
- Public Tunneling Tool ( `pyngrok` )

## 4. Project Workflow & Milestones

### Milestone 1: Model Selection & Architecture

- **Activity 1.1: Generate Groq API Key:** Secure an API key from the Groq console and store it safely in `.env`.
- **Activity 1.2: Research & Model Selection:** Selected `llama-3.3-70b-versatile` for high generation speed, creative output quality, and multi-turn stability.
- **Activity 1.3: Define System Architecture:** Map interactions across the HTML/CSS/JS frontend, Flask server, and Groq LLM API.
- **Activity 1.4: Environment Setup:** Configure virtual environment, project directory structure, and requirements file.

### Milestone 2 & 3: Core Logic Development & app.py Execution

Main application implementation snippet from `app.py` :

```
from flask import Flask, render_template, request, jsonify
from groq import Groq
import os, json
from dotenv import load_dotenv

load_dotenv()
app = Flask(__name__)
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

@app.route("/generate", methods=["POST"])
def generate():
    data = request.get_json()
    product_info = data.get("product_info", "").strip()
    platform = data.get("platform", "instagram").lower()
    tone = data.get("tone", "professional").lower()

    if not product_info or len(product_info) > 2000:
        return jsonify({"error": "Invalid input length"}), 400

    prompt = f"Generate 3 captions, 10 hashtags, 3 CTAs for {product_info} on {platform} in {tone} tone."

    chat_completion = client.chat.completions.create(
        messages=[
            {"role": "system", "content": "Return valid JSON with keys: captions, hashtags, ctas."},
```

```
        {"role": "user", "content": prompt}
    ],
    model="llama-3.3-70b-versatile",
    temperature=0.8,
    max_tokens=2048
)

result = json.loads(chat_completion.choices[0].message.content)
return jsonify({"success": True, **result})
```

## Milestone 4: Frontend Implementation

- **Activity 4.1: Glassmorphism Design:** Clean dark layout featuring subtle glass cards, glowing animated background elements, and FontAwesome icons.
- **Activity 4.2: Asynchronous JS Handling:** Fetch API integration for non-reloading form submit, copy-to-clipboard interactions, and animated loading indicators.

## Milestone 5: Deployment

- **Activity 5.1 & 5.2: Local Server Setup:** Launch server with `python app.py` at `http://127.0.0.1:5050`.
- **Activity 5.3: Public Ngrok Deployment:** Expose Flask port to internet using `run_public.py` and `pyngrok` wrapper for live sharing.

## 5. Conclusion & Future Roadmap

CaptionAI successfully delivers an AI-driven platform tailored for modern social media management. By bringing together Flask, Groq API, and a sleek UI, it enables instantaneous production of polished captions, tags, and calls-to-action.

**Future Scope:** Integration of multi-lingual support, image uploading for automatic visual analysis, and user accounts for prompt history management.